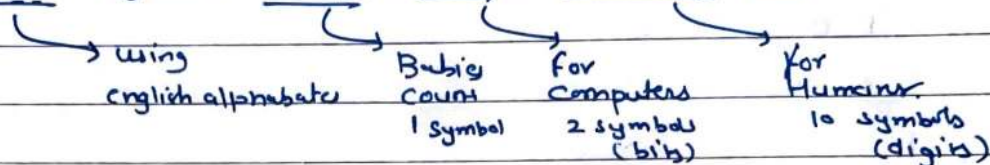


→ M2 in DLD 2024 course

Module-3 Number System

Number SYSTEM: Simply, way to count things or way to represent numbers.

Ex: Roman System, Unary, Binary, Decimal etc.



* Decimal System:

We start to repeat symbols as soon as we finish them.

Ex: In decimal no. system:

0, 1, ..., 7, 8, 9, 10, 11, 12, ..., 20, ..., 99, 100, 101, ..., 999, 1000, ...

Base of no. system is defined as the total no. of symbols available in the no. systems.

So base of decimal system is 10.

Base equals no. of diff values the set has before repeating itself.

Every digit has a weight:

Ex: 2 2 7 0 6
 ↑ ↑ ↑ ↑ ↑
 10^4 10^3 10^2 10^1 10^0

In general,

$(abcde)_x$

(base is x)

$$= (a \times x^4) + (b \times x^3) + (c \times x^2) + (d \times x^1) + (e \times x^0)$$

In $(256)_{10}$ 5 is telling us how many 10's we have skipped and 2 is telling us how many 100's we have skipped.

Control.

DATE : / /

PAGE NO. :

★ Binary System: Also called computer number system.
2 symbols called bits i.e. 0 & 1.

Binary	Decimal
0	0
1	1
1 0	2
1 1	3
1 0 0	4
1 0 1	5
1 1 0	6
1 1 1	7
1 0 0 0	8
1 0 0 1	9
1 0 1 0	10
1 0 1 1	11
1 1 0 0	12
1 1 0 1	13
1 1 1 0	14
1 1 1 1	15

Here in $(1010)_2$, 2nd symbol used after counting
2 items 1 time and 3rd " " " "
4 " 0 times " 4th " " " "
8 " 1 times.

⇒ Binary to Decimal:

$$(1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 \\ = \underline{\underline{(11)_{10}}}$$

★ Binary to Decimal

$$(a_1 a_2 a_3)_2 = a_1 \times 2^2 + a_2 \times 2^1 + a_3 \times 2^0$$

DATE: / /

PAGE NO.:

⇒ Decimal to Binary: To reverse engineering

$$\text{as } (11)_{10} = (1011)_2$$

$$= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \quad - (1)$$

Divide (1) with 2

$$= 1 \times 2^1 + 0 \times 2 + 1 \quad - (2)$$

Modulus of (1) with 2 is last digit of binary

$$\text{So } 11 \% 2 = 1$$

$$\& 11 / 2 = 5 = (3)$$

Now if we divide (3) by 2

$$= 1 \times 2 + 0 \quad - (2)$$

So mod (2) by 2, it's 2nd digit of binary.

$$= 5 \% 2 = 1$$

$$5 / 2 = 2 \quad - (4)$$

And so on we keep on dividing & convert decimal to binary.

Short-handedly:

			→ Remainder (Mod)
2	11	1	
2	5	1	
2	2	0	
2	1	1	
	0		

Reverse Order

$$(11)_{10} = (1011)_2$$

This's how it's working.

DATE: / /

PAGE NO.:

~~Method~~ Remainder obtained at each state is one of the desired bit and least sig. bit is obtained first.

Ex:2 $(156)_{10} = (?)_2$

	Quotient	Remainder
$156 \div 2$	78	0 - <u>LSB</u>
$78 \div 2$	39	0
$39 \div 2$	19	1
$19 \div 2$	9	1
$9 \div 2$	4	1
$4 \div 2$	2	0
$2 \div 2$	1	0
$1 \div 2$	0	1 - <u>MSB</u>

$$= (10011100)_2$$

Generally

$$(abcd)_B = (N)_{10}$$

where B is base of abcd

$$(N)_{10} = (a \times B^3 + b \times B^2 + c \times B^1 + d)_{10}$$

Quotient of N/B is Q_1 $(a \times B^2 + b \times B + c)$

Remainder of N/B is d

Quotient of Q_1/B is Q_2 $(a \times B + b)$

Remainder of Q_1/B is c

Quotient of Q_2/B is Q_3 (a)

Remainder of Q_2/B is b

Quotient of Q_3/B is 0 (0)

Remainder of Q_3/B is a

in base B
we get abcd

DATE: / /

PAGE NO.:

* Octal System: Here we have base 8 i.e. 8 symbols
(0, 1, 2, 3, 4, 5, 6, 7).

0, 1, 2, ..., 7, 10, 11, 12, ..., 17, 20, ..., 77, 100, 101, ..., 777.

$$\begin{aligned}(67)_8 &= (6 \times 8^1 + 7 \times 8^0)_{10} \\ &= (48 + 7)_{10} = \underline{\underline{55}}\end{aligned}$$

* Hexadecimal System: Base is 16.

16 symbols are (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ~~A~~, A, B, C, D, E, F)

$$\begin{aligned}(10F)_{16} &= (1 \times 16^2 + 0 \times 16^1 + 15 \times 16^0)_{10} \\ &= 256 + 15 \\ &= \underline{\underline{(271)_{10}}}\end{aligned}$$

⇒ Base 'r' to Decimal:

for any base r system $r \geq 2, r \in \mathbb{N}$, we have $\#r$ symbols and

$$(abcd)_r = [(a \times r^3) + (b \times r^2) + (c \times r) + (d \times r^0)]_{10}$$

Also if we have radix point

$$(abcd.ef)_r = [(a \times r^3) + (b \times r^2) + (c \times r) + (d \times r^0) + (e \times r^{-1}) + (f \times r^{-2})]_{10}$$

$$\begin{aligned}\text{Ex: } (101.01)_2 &= (1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2}) \\ &= 4 + 1 + \frac{1}{4} \\ &= \underline{\underline{(5.25)_{10}}}\end{aligned}$$

Radix point in decimal no. system is called decimal point.

$$\begin{aligned}\text{Ex: 2 } (11F.0A)_{16} &= (1 \times 16^2 + 1 \times 16 + 15 \times 16^0 + 0 \times 16^{-1} + 10 \times 16^{-2}) \\ &= (256 + 16 + 15 + 0 + \frac{10}{256})_{10} \checkmark\end{aligned}$$

DATE: / /

PAGE NO.:

⇒ Decimal to Base "r" Already seen.

~~N = a~~ Short hand:

$$N = (ar^3 + br^2 + cr + d)$$

~~r/N~~

Divisor	
Quotient	Remainder
$r \mid ar^3 + br^2 + cr + d$	d
$ar^2 + br + c$	c
$ar + b$	b
a	a
0	

So $N = (abcd)_r$

In case of radix point:

$$N = (0.9b)_{10} = (7)_r$$

Let $(0.9b)_{10} = (nyz)_r$

$$N = (0.9b)_{10} = \left(\frac{x}{r} + \frac{y}{r^2} + \frac{z}{r^3} \right)_{10}$$

Now $N \times r$

$$= \frac{x}{r} + \frac{y}{r} + \frac{z}{r^2}$$

fractional/Decimal part

Integer part = x

Now ② $\times r$

$$= \frac{y}{r} + \frac{z}{r}$$

fractional/Decimal part

Integer part = y

Now ③ $\times r$

$$= \frac{z}{r}$$

Integer part.

DATE: / /

PAGE NO.:

Ex:1 $(.152)_{10} = (?)_8$

$$\begin{aligned} .152 \times 8 &= 1.216 \\ .216 \times 8 &= 1.728 \\ .728 \times 8 &= 5.824 \\ .824 \times 8 &= 6.592 \end{aligned}$$

$$= (.152)_{10} = (.1156\ldots)_8$$

Ex:2 $(.25)_{10} = (?)_2$

$$\begin{aligned} .25 \times 2 &= 0.5 \\ .5 \times 2 &= 1.0 \\ &= (.01)_{20} \end{aligned}$$

Ex:3 $(.3)_{10} = (?)_2$

$$.3 \times 2 = 0.6$$

$$.6 \times 2 = 1.2$$

$$.2 \times 2 = 0.4$$

$$.4 \times 2 = 0.8$$

$$.8 \times 2 = 1.6$$

$$.6 \times 2 = 1.2$$

← Repetition.

$$= (.01001)_2$$

→ keep repeating.

Basically for decimal to base conversion we handle decimal & fraction separately.

for Ex: $(10.625)_{10} = (?)_2$

2	10	0
2	5	1
2	2	0
2	1	1
	0	

$$.625 \times 2 = 1.250$$

$$.25 \times 2 = 0.5$$

$$0.5 \times 2 = 1.0$$

$$\text{So, } = (1010.101)_2$$

$$= (1010.101)_2$$

DATE: / /

PAGE NO.:

Ques:

$$(23.3)_4 \rightarrow (?)_7$$

$$\Rightarrow (23.3)_4 = (2 \times 4^1 + 3 \times 4^0 + 3 \times 4^{-1} + 1 \times 4^{-2})_{10}$$

$$= (11.8125)_{10}$$

$$(11.8125) = (?)_7$$

7	11	4
7	1	1
	0	

$$.8125 \times 7 = 5.6875$$

$$.6875 \times 7 = 4.8125$$

$$.8125 \times 7 = 5.6875$$

$$= (14.54)_7$$

Ans.Ques:

Why octal & hexadecimal systems? What's is its importance?

$\Rightarrow$ As we know need of binary in computer system (High & Low Voltage).

But, binary is a long sequence. ~~Ques~~ Binary can be easily converted to ^{octal} binary by grouping 3 bits each as $(2^3 = 8)$

Ex: $(011\ 011\ 010\ 101\ 100\ 101)_2 = (332545)_8$

Similarly hexadecimal can also be got from binary.

Ex: ^{append 200} $(0100101010110111)_2 = (8A77)_{16}$

step 1/4

Always, start to make groups from left of radix to right of radix, and append any no. of zeroes to complete grp

Ex: $(\text{0000} \cdot \text{bcde} \cdot \text{f900})_2 = (?)_{16}$

$\swarrow$ $\downarrow$ $\searrow$
 Grp2 Grp1 Grp1

As binary can be converted back easily from octal to hexadecimal. we convert decimal to octal/hd. then # steps & then easily get binary from octal/hd.

$$(Src)_{10} \rightarrow (mid)_{8/16} \rightarrow (Dest)_2$$

So, basically we reduce # steps to this way beneficial for computers.

Append zero

$$(101 \cdot 1011)_2 \Rightarrow (00101 \cdot 101100)_2 = (15.54)_{10}$$

Binary Arithmetic: Similar to decimal arithmetic of schools.

Addition

if digit $n+y < \text{base}$.

write as it is

if digit $n+y \geq \text{base}$

write $\text{base} - (n+y)$ & 1 as carry

Ex:

	1		1	
	2	3	0	4
	9	8	7	6
	12	1	0	0

$9+2+1 \Rightarrow 12$
 $8+3=11$
 $1+0+7 < \text{Base}$

$10 \geq \text{Base}$
 0 & 1 carry
 1 carry, 1 written

DATE: / /

PAGE NO.:

Similarly for Binary:

$$\begin{array}{r}
 1111 \\
 101101 \\
 \hline
 110111 \\
 \hline
 1100100
 \end{array}$$

$2 > \text{Base}$
 write $2-2$ as 1 carry
 $1+1+1=3 > \text{Base}$
 write $3-2$ as 1 carry

In worst case we have to add $1+1+1=3$ so 1 carry & sum.

	x	y	Carry _{in}	Sum	Carry _{out}
	0	0	0	0	0
	0	0	1	1	0
	0	1	0	1	0
	0	1	1	0	1
	1	0	0	1	0
	1	0	1	0	1
	1	1	0	0	1
	1	1	1	1	1

So addition is Easy-Peasy.

★ Subtraction:

Subtraction were awesome if we don't have to borrow.
We can guarantee No Borrow if we subtract from all 9s.

Ex:
$$\begin{array}{r} 999 \\ - 784 \\ \hline 215 \end{array}$$
 No borrow

Another way to subtract.

$$\begin{aligned} 82 - 64 &\Rightarrow 82 + 100 - 100 - 64 \Rightarrow 82 + (100 - 64) - 100 \\ &\Rightarrow 82 + (99 - 64) - 100 + 1 \Rightarrow 82 + (99 - 64) - 100 + 1 \end{aligned}$$

Easy subtraction as No borrow

$$\Rightarrow 82 + 35 - 100 + 1 \Rightarrow 118 - 100 = 18$$

Addition is any ways easy. ignore carry.

This subtraction is done by complementing the number.
So that we don't have to borrow.

$\Rightarrow$ Complements of a number in base 'r'.

(r-1)'s complement

$\downarrow$
Diminished radix
Complement

r's Complement

$\downarrow$
Radix
Complement.

In decimal we have 9's & 10's complement.
" Octal " " 7's & 8's " "
" Binary " " 1's & 2's "

$$\frac{1}{\frac{1}{b}} \leftarrow \left\{ \frac{1}{\frac{1}{b}}, \frac{1}{\frac{1}{b}} \right\}$$

DATE: / /

PAGE NO.:

If we have n digits in no. N of base r
 then $(r-1)$'s complement is $(r^n - 1) - N$ and
 r 's complement is $r^n - N$.

Ex: $N = (82)_{10}$

$$\begin{aligned} 9\text{'s comp} &= 100 - 1 - 82 \\ &= 99 - 82 \\ &= \underline{17} \end{aligned}$$

$$\begin{aligned} 10\text{'s comp} &= 10^2 - 82 \\ &= 100 - 82 \\ &= \underline{18} \end{aligned}$$

Note $\leftrightarrow$ r 's complement = $(r-1)$'s complement + 1 and.
 in $(r-1)$'s comp. $\underbrace{r \times r \times \dots \times r}_{n \text{ times}} - N$ ✓

Ex: 2 $N = (0101101)_2$

$$\begin{aligned} 1\text{'s comp} &= 11111 - 0101101 \\ &= 1010010 \end{aligned}$$

$$\begin{aligned} 2\text{'s comp} &= 1\text{'s comp} + 1 \\ &= 1010010 + 1 \\ &= 1010011 \end{aligned}$$

In binary $1 - x$ is $\bar{x}$ So, to get 1's complement
 just flip the bits.

Ex: 1's comp. of $(110110100)_2$ is $(001001011)_2$ ✓

In r 's complement

let $N = (\underbrace{edcba}_{\text{anything}} \underbrace{000}_{\text{Non-zero}})_r$

then r 's comp. : $(r-1-e)(r-1-d)(r-1-c)(r-1-b)(r-1-a)000$

Ex: 10's comp. of $(8201129400)_{10}$
is 1798870400

Ex: 2's comp of $(11010010)_2$
is $(0010110)_2$ $\rightarrow$ as it's

Shortcut for 2's comp is put all no. till first 1 as it's 1 then complement other bits

$\Rightarrow$ Subtraction of unsigned no. using r 's complement

Let M, N be unsigned no.

$$M - N$$

Case 1 $M \geq N$

Ex: $82 - 64 = 82 + (99 - 64) + 1 - 100$
 $\hookrightarrow$ No borrow

$$= 82 + 35 + 1 - 100$$

$$\Rightarrow 82 + 36 - 100$$

$\downarrow$
10's comp.

$$\Rightarrow 118 - 100 \quad (\text{delete trailing 1.})$$

$$\Rightarrow \underline{18}$$

So if $M \geq N$, then we discard end carry.

M

+ 10's comp of N

result

Carry

Discarded

DATE: / /

PAGE NO.:

Case 2: $M < N$

Ex: $20 - 78 = 20 + (99 - 78) + 1 - 100$
 $\Rightarrow 42 - 100$
 $= \cancel{42} - (100 - 42)$
 $= -58.$ $\rightarrow$ 10's comp. of 42.

So no end carry in $M < N$

$$\begin{array}{r} M \\ - N \\ \hline \text{result} \end{array}$$

$\rightarrow$ Correct answer will be
 $- (10's \text{ comp. of result})$

Summary: M, N are unsigned no, r is base to use
 want $M - N$ using r 's complement.
 We find $M + (r's \text{ comp. of } N)$
 $= M + (r^n - N)$
 $= M - N + r^n$

Case 1: $M > N \rightarrow$ End carry Exist

Discard it & you have answer.

Case 2: $M < N \rightarrow$ End carry don't exist $- (r's \text{ comp. of } (M - N + r^n))$ & this is answer.

Ex: $239 - 82$
 $\Rightarrow 239 + 918$ $\rightarrow$ 10's comp. of 82
 $\Rightarrow 1157$
 $\therefore$ End carry
 $= 157$
Ans.

Ex 2: $82 - 239$
 $\Rightarrow 82 + 761$ $\rightarrow$ 10's comp. of 239
 $\Rightarrow 843$
 $\therefore$ No end carry
 $\Rightarrow -157$ $\rightarrow$ 10's comp. of 843
Ans.

Now in Binary

Ex:1
$$\begin{array}{r} 1111010 \\ - 1101011 \\ \hline \end{array}$$

$$\Rightarrow \begin{array}{r} 1111010 \\ + 0010101 \quad \leftarrow \text{2's comp of } 1101011 \\ \hline 10001111 \\ \hline \end{array}$$

end carry $\rightarrow$ discard $\rightarrow$ ans. is 0001111 ✓

Ex:2
$$\begin{array}{r} 1101011 \\ 1111010 \\ \hline \end{array}$$

$$\Rightarrow \begin{array}{r} 1101011 \\ + 0000110 \\ \hline 1110001 \\ \hline \end{array}$$

No end carry $\rightarrow$ ~~0001111~~ $\rightarrow$ - 0001111 ✓

$\Rightarrow$ Subtraction of unsigned no. using (x-1)'s complement.

Ex: $82 - 24 \Rightarrow 82 + (100 - 24) - 100$

$\Rightarrow 82 + (99 - 24) + 1 - 100$

$\Rightarrow 82 + (9\text{'s comp of } 24) + 1 - 100$

$\Rightarrow 82 + 75 + 1 - 100$

$\Rightarrow 82 + 76 - 100$

$\Rightarrow 158 - 100$

$\Rightarrow$ 58

If $m \geq n$, we have end carry, we add it (i.e. add 1) & discard the carry.

If $m < n$, we don't have end carry, we just 9's comp it again.

Ex: $24 - 82 = 24 + (9\text{'s comp of } 82) + 1 - 100$

$\Rightarrow 24 + 17 + 1 - 100$

$\Rightarrow (41) - 99$

$\Rightarrow - (99 - 41) \Rightarrow - (9\text{'s comp. of } 41)$ ✓

DATE: / /

PAGE NO.:

Summary: If M, N are unsigned no., r is base & we want $M - N$ using r 's complement

We find $M + (r-1)$'s comp. of N

$$= M + (r^n - 1 - N)$$

$$= (M - N) + (r^n - 1)$$

Case 1: $M \geq N \rightarrow$ End carry

$\rightarrow$ Add 1 (or End carry) & discard carry. Ans

Case 2: $M < N \rightarrow$ No End carry

$\rightarrow$ - (9's comp. of $(M - N) + (r^n - 1)$)

Ex:

$$\begin{aligned} & 239 - 82 \\ \Rightarrow & 239 + 917 \\ \Rightarrow & 1156 \quad (\text{Add end carry}) \\ \Rightarrow & 156 + 1 \\ \Rightarrow & \underline{\underline{157}} \end{aligned}$$

Ex:2

$$82 - 239$$

$$\Rightarrow 82 + 760$$

$$\Rightarrow 842 \quad (\text{No end carry})$$

$$\Rightarrow -(157) \xrightarrow{\text{9's comp of 842}}$$

Basically for r 's & $(r-1)$'s comp. do not memorize steps, try them with decimals by your own & make your own rules.

(Easy hai, looks hard).

Now in Binary (using 1's complement)

Ex 1: $1101 - 0011$

$\Rightarrow 1101 + 1100$
1's comp. of 0011

$\Rightarrow 10001$
End carry

$\Rightarrow 0001 + 1$

$\Rightarrow 0010$ ✓

Ex 2

$0011 - 1101$

$\Rightarrow 0011 + 0010 \rightarrow 1's \text{ comp. of } 1101$

$\Rightarrow 0101$
No end carry

$\Rightarrow -1101$
1's comp. of 0101 ✓

★

r's or (r-1)'s complement for Radix Point.

Same method:

Ex 1:

$(39.502)_{10}$

10's
 $\swarrow$
 60.498

9's Complement
 $\searrow$
 60.497

Ex 2:

$(10100.1011)_2$

2's $\swarrow$ 1's comp.
 $\searrow$

01011.0101

01011.0100

$\Rightarrow$ Complement is needed bcz "sub. with borrow" is hard & addition is easy (for humans) & in frn h/w pay, we will be able to do all work by single h/w only i.e. adder. (No subtraction diff. h/w req.).

DATE: / /

PAGE NO.:

* Representations of Signed Binary Numbers: only for Binary.

Decimal = +199 - 199

→ How to write it to D.C. using 0&1 only.

Signed Binary No.

Sign-Magnitude System

Sign. Complement System

1's Comp. System

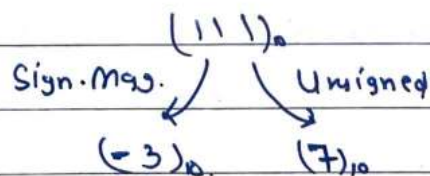
2's Comp. System

a). Signed Magnitude System: Use extra bit to store sign
 $0 \rightarrow +$ & $1 \rightarrow -$

for Ex: 0100 means +4 & 1100 means -4. in
 Sign. mag. system.

9 in sign. mag. system with 4 bits only is
NOT possible. It will be 01001.

↓
Mandatory



→ 0 in Sign. mag. has two rep. 00 & 10.

b) 1's Complement No. System:

$$\text{SM System} \equiv \text{1's Comp. System} \equiv \text{2's comp. System}$$

↳ for $n \geq 0$.

for $n < 0$, in 1's comp. system $(-N)$ is stored as 1's complement of N .

Ex: 9 in 1's comp. system 01001
 -9 " " " " (01001)'s 1's comp = 10110.

Here also MSB is 1 for -ive no. always like SM system.

Ques: Repr. -8 in 4 bits in 1's comp. system.
 ⇒ (1000)'s 1's comp = 0111 } Mistake.
 ↳ It is +7 in 1's comp. system.

⇒ Can't Rep. bcz

$$+8 = (01000) \rightarrow \text{Req. 5 bits.}$$

Here 0 can be repr. as 00 or 11 (1's comp of +0).

c) 2's Complement Number Systems:

for $n < 0$, in this no. system $(-N)$ is stored as 2's complement of N .

Ex 9 in 2's comp 01001
 -9 " " " " 10111

Here also MSB is 1 for -ive no. & 0 for +ive no.

1's comp. of 1's comp is actual no.
 2's " " 2's " " " "

DATE: / /

PAGE NO.:

So, In all systems +ve integers have same binary, but for -ve no. they differ.

Ques: What do following no. repr:

a) $(01010)_2 \Rightarrow \underline{(10)_{10}}$

b) $(11010)_2 \Rightarrow$ Depends on no. system used.

→ If unsigned: 26

→ If signed

→ Sign mag. = -10

→ 1's comp. no. system: -5

→ 2's comp. no. system: -6

NOTE → 1's comp. operation is a unary opⁿ on any no.
 2's comp. no. system is a storage system to store signed numbers. Don't confuse b/w them.

Ex: $00101 \xrightarrow{1's \text{ comp}} \underline{\underline{11010}}$
 What it repr in 2's no. sy. $\rightarrow \underline{\underline{+5}}$

NOTE → 2's comp. no. system is weighted no. system (i.e. every position is associated some weight). Like binary, decimal no. system.

In 2's comp. no. system

$$abcde := -(a \times 2^4) + (b \times 2^3) + (c \times 2^2) + (d \times 2^1) + (e \times 2^0)$$

Ex: $10110 = -2^4 + 2^2 + 2^1$
 $\Rightarrow \underline{\underline{-10}}$
 In 2's comp. no. system

DATE: / /

PAGE NO.:

So in 2's comp. no. system MSB has negative weight & all other bits have positive weight. (You know it!).

Also, in 2's comp. system extending MSB do not change it.

$$\text{Ex: } 00011 = 00011 = 000011 = 011 = 3.$$

$$101 = 1101 = 11101 = \underbrace{11111}_{\text{Don't Matter}} 101 = -3 \quad \checkmark$$

→ Sign-mag. & 1's comp. No. system are non-weighted.

→ With 3 bits we should be able to repr. 8 diff. no. but with sm & 1's comp. no. sys. we are able to repr. only 7 diff. no. bcz '0' has got two representations. So 1 comb. is wasted in sm & 1's comp. no. system.

	Unsigned	Sign-Mag.	1's comp.	2's comp.	Same in all
0 0 0	0	0	0	0	}
0 0 1	1	1	1	1	
0 1 0	2	2	2	2	
0 1 1	3	3	3	3	
1 0 0	4	-0	-3	-4	}
1 0 1	5	-1	-2	-3	
1 1 0	6	-2	-1	-2	
1 1 1	7	-3	-0	-1	

$$\text{In sign-mag} = 0000 = 1000 = (0)_{10}$$

$$\text{In 1's comp} = 0000 = 1111 = (0)_{10} \quad \checkmark$$

In 2's comp. No. every no. has unique representation.

DATE: / /

PAGE NO.:

★ Range of diff. no. systems:

a) Unsigned : for 3 bits : 0 to 7
for n bits : 0 to $2^n - 1$

b) S.M.Ns : for 3 bits : -3 to 3
for n bits : -2^{n-1} to $2^{n-1} - 1$

c) 1's comp Ns : for 3 bits : -3 to 3
for n bits : $-2^{n-1} + 1$ to $2^{n-1} - 1$

d) 2's comp Ns : for 3 bits : -4 to 3
for n bits : -2^{n-1} to $2^{n-1} - 1$

	in sm.	in 1's comp	2's comp.	Unsigned
Min.	11111111	10000000	10000000	00000000
Max.	01111111	01111111	01111111	11111111

⇒ Now, we have already seen addition & subtraction in case of unsigned numbers.

SubtractionAddition of Signed Binary Numbers.

Q. No. in Signed Magnitude System: (Addition)

Similar to Decimal Addition we do

i.e. $(+x) + (+y) = +(x+y)$

$(-x) + (-y) = -(x+y)$

$(+x) + (-y) = -(\overbrace{x-y}^{y-x})$

if $|y| > |x|$ if $|x| > |y|$

choose sign from greater mag. no.

Ex: $12 + 13 = 25$; $12 - 13 = -1$; $13 - 12 = 1$; $-13 - 12 = -25$.

Similarly, for Binary No.

Compare sign bit

a1: both -ive, $\rightarrow$ add both and put -ive symbola2: both +ive $\rightarrow$ add both & put +ive symbola3: both diff $\rightarrow$ Compare Magnitudesubtract bigger from lesser
to use symbol of higher magnitude.

Ex: $(-3) \quad (-7)$
 $1011 + 0111$

diff symbol $111 > 011$

$$\begin{array}{r}
 111 \\
 - 011 \\
 \hline
 100 \\
 \hline
 \Rightarrow 4
 \end{array}$$

$\Rightarrow$ 0100 result

Note that all i/p will be in same no. system & o/p will also be in same no. system

DATE: / /

PAGE NO.:

b) Subtraction of Signed no. in sm system.

$$A - B \Rightarrow \underline{\underline{A + (-B)}}$$

Ex:

$$\begin{array}{r} -3 \quad 7 \\ 1011 - 0111 \end{array}$$

$$\Rightarrow 1011 + 1111 \quad (\text{Change symbol})$$

$$\Rightarrow \quad \hookrightarrow \text{Addition of no. in sm system}$$

$$\Rightarrow 1(011 + 111)$$

Sign bit $\Rightarrow$

$$\underline{\underline{11010}} \quad \underline{\underline{-10}}$$

Cost:

Sign bit Comparison

It diff then mag. Comparison to Subtraction

It same then addition

 $\Rightarrow O(n)$ time to Addition + Subtraction hardware.c) Addition of sig. no. in 2's Complement system:

Add. numbers normally to discard carry

$$\underline{\underline{\text{Ex:1}}} \quad \begin{array}{r} 00101 \quad (5) \\ + 11100 \quad (-4) \\ \hline 100001 \end{array}$$

$$\begin{array}{r} 100001 \\ \hline \end{array} \Rightarrow \underline{\underline{0001}} \quad (1)$$

Discard

$$\underline{\underline{\text{Ex:2}}} \quad \begin{array}{r} 11011 \quad (-5) \\ + 11100 \quad (-4) \\ \hline 11011 \end{array}$$

$$\begin{array}{r} 11011 \\ \hline \end{array} \Rightarrow \underline{\underline{1011}} \quad (-9)$$

Discard

$$\underline{\underline{\text{Ex:3}}} \quad \begin{array}{r} 00101 \quad (5) \\ + 11010 \quad (-6) \\ \hline 11111 \quad (-1) \\ \hline \end{array}$$

No carry

Basically ifp is 2 no. in 2's comp. no. system, add them & discard carry. O/p is sum in 2's comp. no. system.

Logic for 2's comp system $\vec{M}, \vec{N}$

Made By - Karan Agrawal (GATE CS 2024 AIR 102)

$$\Rightarrow (2^m - M) + (2^N - N) \\ = [2^m - (M+N)] \rightarrow \text{sum } M+N \text{ in } 2\text{'s comp.}$$

Carry $\leftarrow$

DATE: / /

PAGE NO.:

d) Addition of 2 sig no in 1's comp system: Add the no. and

also add the carry. (if exist)

Ex:

$$\begin{array}{r} 00101 \quad (5) \\ 00100 \quad (4) \\ \hline 01001 \quad (5) \end{array}$$

Ex 2:

$$\begin{array}{r} 00101 \quad (5) \\ 11011 \quad (-4) \\ \hline 10000 \\ \rightarrow +1 \\ \hline 0001 \quad (1) \end{array}$$

Ques: Add -2 & 19 in

a) Sign mag. system

$\Rightarrow M = 101000$

$N = 010011$

$M+N = +(N-M)$

$= 01011$

b) In 2's comp. system

$\Rightarrow M = 111000$

$N = 010011$

$M+N = 1001011$

Discard

$M+N = 001011$

c) In 1's comp system

$\Rightarrow M = 1110111$

$N = 010011$

$M+N = 1001010$

$\rightarrow +1$

$M+N = 001011$

d) Unsigned system

$\Rightarrow$ NON SENSE

DATE: / /

PAGE NO.:

e) Subtraction of two no. in 2's complement system:

If A & B are in 2's comp. system
then $A - B = A + 2\text{'s comp of } B$.

Discard the end carry.

Ex: 1 $+4: 00100 = A$
 $+9: 01001 = B$
 $A - B = A + (B)\text{'s 2's comp}$
 $=$

$$\begin{array}{r} 00100 \\ + 10111 \\ \hline 11011 \end{array} = \underline{\underline{-5}} \checkmark$$

Ex: 2 $-4 = 11100 = A$
 $+9 = 01001 = B$
 $A - B$
 $= A + 2\text{'s comp. of } B$

$$\begin{array}{r} 11100 \\ + 10111 \\ \hline 10011 \end{array}$$

Discard

$$= \underline{\underline{0011}} \quad -13 \checkmark$$

f) Subtraction of two no. in 1's complement system:

If A & B are in 1's comp. system

then $A - B = A + 1\text{'s comp of } B$

and wrap around the carry.

DATE: / /

PAGE NO.:

Ex:1 $A = 11011 = -4$

$B = 11010 = -5$

$A - B = A + 1's \text{ comp. of } B$

$$\begin{array}{r}
 11011 \\
 00101 \\
 \hline
 100000 \\
 \leftarrow \text{wrap around carry} \\
 \hline
 00001 \quad \checkmark
 \end{array}$$

Ex:2 $A = 00100 = 4$

$B = 11010 = -5$

$A - B = A + 1's \text{ comp. of } B$

$$\begin{array}{r}
 00100 \\
 00101 \\
 \hline
 01001 \quad \checkmark
 \end{array}$$

→ So, in signed comp. system (1's & 2's both) addition & subtraction is done using single same circuit i.e. added & also no. comp. required.

→ 2's complement system is better than 1's complement no. system for signed numbers bcz:

a. No need to wrap around carry

b. No two rep. of 0.

c. Better range

★ Overflow for Addition operation: We store bits in registers in computer.

Size of registers is fixed. i.e. it can have ~~up to~~ n bits only.

when two n bits data is operated we may get result of n+1 or more bits of data, but register can only store n bits. So in this case MSB

OVERFLOW

Rocket Crashed $\rightarrow$ due to overflow.

Made By - Karan Agrawal (GATE CS 2024 AIR 102)

DATE: / /

PAGE NO.:

Ex: We have 4 bit registers

we add two unsigned no. $10 \rightarrow (1010)_2$ & $13 \rightarrow (1101)_2$
result is 23 which need 5 bits to be stored.
which leads to overflow.

Overflow depends on:

- $\rightarrow$ Representation of Data (Unsigned, Signed, etc)
- $\rightarrow$ Operation (Addition, Sub, mul, etc)

NOTE $\leftrightarrow$ Only addition don't cause overflow, other operations can also do. BUT, in GATE syllabus we study overflow for addition only, in unsigned and signed (all 3 no. system) representations.

A) Overflow in case of unsigned addition: Overflow def. is same, just the detection condⁿ changes acc. to opⁿ (addition) & representation.

"If carry comes out of MSB" $\Rightarrow$ Overflow
 $\uparrow$
Left Bit

Ex: $1 + 1 = 10$
 $\downarrow$
Overflow for 1 bit register.

Ex: 2 bits. (Range 0 to 3)

$2 + 2 \rightarrow$ OF

$2 + 1 \rightarrow$ NO OF

$-3 + -4 \rightarrow$ Non sense

$4 + 1 \rightarrow$ Non sense

$\nwarrow$
Already out of range

DATE: / /

PAGE NO.:

So In unsigned no. for let's say 4 bits

$$A = a_3 a_2 a_1 a_0$$

$$B = b_3 b_2 b_1 b_0$$

$$\underline{\underline{r_4 \ r_3 \ r_2 \ r_1 \ r_0}}$$

OF occurs iff $r_4 = 1$

B) Overflow in case of signed no. addition:

Imp: Overflow can NEVER occur if numbers have opposite sign. (Obvious)

Bl Overflow in sign magnitude addition: If carry from magnitudes' Sum.

Let ~~a_4~~

$$A = s_a a_1 a_2 a_3 a_4$$

$$B = s_b b_1 b_2 b_3 b_4$$

If $s_a \neq s_b$ OF can't happen.

If $s_a = s_b$, then OF occurs iff magnitudes of both gives carryout of MRS.

Ex:-
$$\begin{array}{r} 1110 + 0011 \quad [-6 + (+3)] \\ \hline \end{array}$$

 $\hookrightarrow$ No OF as $s_a \neq s_b$

Ex2:
$$\begin{array}{r} 1110 + 1011 \quad [-6 + (-3)] \\ \hline \end{array}$$

 $\hookrightarrow s_a = s_b$

$$\begin{array}{r} 110 \\ 011 \\ \hline 1001 \end{array}$$

Carry out $\rightarrow$ Overflow

B2 OF in 2's Compt. addition: There is no overflow if:

- For both 1's & 2's Complement sum
- Both operand has ~~same~~ ^{diff} msb (i.e. diff signs), OR
 - Both operand are same i.e. sum is same, OR
 - " " " " -ive " " " -ive.

OF if:

$$\begin{array}{r}
 +A \\
 +B \\
 \hline
 -R
 \end{array}
 \quad \text{OR} \quad
 \begin{array}{r}
 -A \\
 -B \\
 \hline
 +R
 \end{array}$$

Overflow

Ex:
$$\begin{array}{r}
 0100 \quad (4) \\
 + 0100 \quad (4) \\
 \hline
 1000 \quad (-8)
 \end{array}$$
 type type -ive
So overflow

OF condition:

$$\boxed{Q_{msb} \oplus b_{msb} \oplus S_{msb} + Q_{msb} \oplus b_{msb} \oplus S_{msb} = 1}$$

msb of sum

→ If true, then OF.

Ex:2
$$\begin{array}{r}
 1101 \quad (3) \\
 1100 \quad (4) \\
 \hline
 11001 \quad (7) \text{ No OF.}
 \end{array}$$
 Discard

Ex:3
$$\begin{array}{r}
 1011 \quad (-5) \\
 1010 \quad (-6) \\
 \hline
 0101 \quad (5) \text{ OF}
 \end{array}$$
 Discard

We have to discard carry, it's a part of addition.

Always check OF in final result i.e.
discard Cout in 2's comp &
wrap around Cout in 1's comp.

DATE: / /

PAGE NO.:

B3 OF in 1's Comp. Addition

Same Three Conditions of No Overflow

Ex:1

0101	(5)	<u>NO OF</u> ↳ Also diff sign
1001	(-6)	
1110 ✓	(-1)	

Ex:2

1010	(-5)	<u>NO OF</u>
0110	(+6)	
10000		
↳ 1		
0001	(1)	

Ex:3

1010	(-5)	<u>OF</u>
1011	(-6)	
10011		
↳ 1		
0100	(4)	

Here due,

$$\overline{A_{msb}} \oplus \overline{B_{msb}} \oplus \overline{C_{out}} = 1$$

↳ OF

An alternative method for detecting overflow in 2's comp addition is as: "An OF occurs iff Cout of the MSB is not equal to Cin of MSB".

Bcz Consider only MSBs

$\begin{array}{r} \text{Cout} \nwarrow \swarrow \text{Cin} \\ 1 \quad 0 \\ \hline 0 \quad 1 \\ \hline \end{array}$ OF can't occur	$\begin{array}{r} \text{Cout} \nwarrow \swarrow \text{Cin} \\ 1 \quad 0 \\ \hline 1 \quad 1 \\ \hline \end{array}$ <u>OF</u>	$\begin{array}{r} \text{Cout} \nwarrow \swarrow \text{Cin} \\ 0 \quad 1 \\ \hline 0 \quad 0 \\ \hline \end{array}$ <u>NoOF</u>	$\begin{array}{r} \text{Cout} \nwarrow \swarrow \text{Cin} \\ 0 \quad 0 \\ \hline 1 \quad 1 \\ \hline \end{array}$ <u>OF</u>
--	---	---	---

So OF iff Cin & Cout are diff

So ~~OF~~ iff Cin ⊕ Cout = 1

in 2's comp. only

DATE: / /

PAGE NO.:

★ Sign Extension: Extra bits to fill when copying content of n bit register to $>n$ bit register. So that value remains same

a) For unsigned numbers: Copy zeros

$$1101 = 01101 = \underbrace{00}_{\text{any zeros}} \dots 01101$$

$$\text{So } \underbrace{a_n a_{n-1} \dots a_1}_{n \text{ bits}} \xrightarrow{\text{Copy}} \underbrace{00 \dots 0}_{\#n} a_n a_{n-1} \dots a_1_{n+n \text{ bits}}$$

b) For signed no. in sign magnitude repr.:

$$01001 = 001001 = 00 \dots 01001$$

$$11001 = 1001001 = \underbrace{10}_{\text{any zeros}} \dots 01001$$

$$\text{So } \underbrace{a_n a_{n-1} \dots a_1}_{n \text{ bits}} \xrightarrow{\text{Copy}} \underbrace{a_n \underbrace{00 \dots 0}_{\#n} a_{n-1} a_{n-2} \dots a_1}_{n+n \text{ bits}}$$

c) For signed no. in 1's Comp. repr.: Copy MSB

$$0110 = 00110 = 0 \dots 0110$$

$$1001 = 11001 = 1 \dots 1001$$

$$\text{So } \underbrace{a_n a_{n-1} \dots a_1}_{n \text{ bits}} \xrightarrow{\text{Copy}} \underbrace{a_n \overbrace{a_n a_{n-1} \dots a_{n-1}}^n a_{n-1} a_{n-2} \dots a_1}_{n+n \text{ bits}}$$

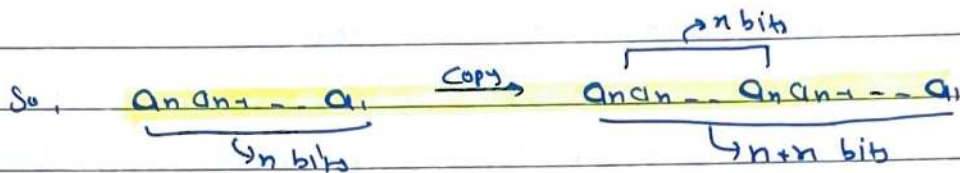
DATE: / /

PAGE NO.:

Q) for signed no. in 2's comp. repr. Copy MSB

$$0011 = 00 \dots 011$$

$$1011 = 11 \dots 1011$$



Binary Codes: Code in general is a system of rules to convert information - such as letter, word, image or gesture into another repr. sometimes to store or secret or for making it suitable to travel on channel, etc.

Binary codes are pattern of 0's & 1's only.
i.e. code containing 0 & 1 only.

Ex: abc can be written as 00001000

For n diff elements we need at least $\lceil \log_2 n \rceil$ bits.

i.e. k bits can give 2^k diff elements.

There are diff binary codes, each having its own advantages.

Binary codes are imp. to store data in computer/ckts.

We study binary codes despite having binary no. (which is also a binary code) bco, we can't repr. letters, symbols in binary no., we need some coding. Also have many application like error control, enc., etc.

Ascii of 'A' is 65 - 90

of 'a' is 97 - 122

of '0' is 48 - 57

DATE: / /

PAGE NO.:

Binary codes for decimal digits are:

Decimal Digit	2-out-of-5 code	Excess 3 code	8421 Binary Coded Decimal (BCD)	2421 code	84-2-1 code
0	00011	0011	0000	0000	0000
1	00101	0100	0001	0001	0111
2	00110	0101	0010	1000, 0010	0110
3	01001	0110	0011	1001, 0011	0101
4	01010	0111	0100	1010, 0100	0100
5	01100	1000	0101	1011	1011
6	10001	1001	0110	1100	1010
7	10010	1010	0111	1101	1001
8	10100	1011	1000	1110	1000
9	11000	1100	1001	1111	1111

Provider bit error detecting i.e. 2 Hamming dist. (for 2-out-of-5 code)
 SCC code (BCD) for each digit (for 2421 and 84-2-1 codes)
 Unsigned digit also (for 2421 and 84-2-1 codes)
 SCC code & weighted code (for 84-2-1 code)

49 in BCD is 0100 1001 & in binary is 110001

→ Unused comb. of BCD are referred invalid.

→ BCD No. is Decimal no. represented using bits only. It is not binary number.

→ BCD is also known as 8421 code. i.e. weight at every position. Ex: $0101 = 4 + 1 = 5$

→ In 2421 Code. Some no. like 2 has 2 possible encodings i.e. 1000 & 0010, but we prefer 0010 i.e. will have decimal eq. if treated as binary till 4 & after that use self complementing property.

→ Self Complementary Code: when in decimal x to $9-x$ has codes y in and its comp. of y . those

codes are SCC.

Ex: 2421 code

↳ 2 is 0010

9-2 i.e. 7 is 1101

BCD codes are not SCC.

↳ 2 is 0010

9-2=7 is not 1101

→ Weighted codes has weight at every position.

→ Excess 3 code are also SCC. but unweighted.

$(5)_{10} = (0101)_2 \xrightarrow{+3} 1000$

$(9)_{10} = (1001)_2 \xrightarrow{+3} \underline{\underline{1100}}$

→ BCD, 2421, 84-2-1, Excess 3, 2-out of 5, there are all per decimal digit codes. And each has its own advantage
 bcz. for each digit → fix code

→ If a code is SCC, then sum of weight is 9.

SCC → Sum of weight 9.



★ Gray Code: It has many applications such as counter but here with each next count only 1 bit changes.

Ex: 2 → 3

10 → 11

↳ So only need to flip one bit.

This is main property of gray code.

→ Gray codes are not per digit decimal codes.

→ Gray Codes $\Rightarrow$ Reflected Binary Codes (RBC) $\Rightarrow$ Reflected Binary (RB)
 $\swarrow$ due to this

★ Rec. Construction of Gray codes

Let G_n : Gray code with n bits

G_1 :	G_2 :	G_3 :	Decimal
0	0 0	0 0 0	0
1	0 1	0 0 1	1
	1 1	0 1 1	2
	1 0	0 1 0	3
		1 1 0	4
		1 1 1	5
		1 0 1	6
		1 0 0	7

Here for G_n write G_{n-1} , and then reflection of G_{n-1} then add 0 in starting G_{n-1} and 1 at reflected G_{n-1} .

But recursive method might be time consuming.

So we have efficient method to convert decimal/binary to gray & vice versa.

⇒ Binary No. to Gray Code.

Let binary m :



Gray Code f_m :

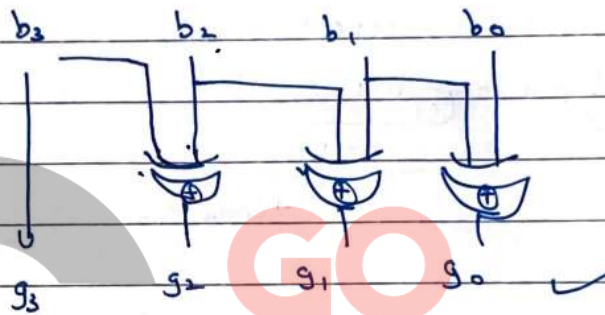
$$b_3 \quad b_2 \oplus b_3 \quad b_1 \oplus b_2 \quad b_0 \oplus b_1$$

DATE: / /

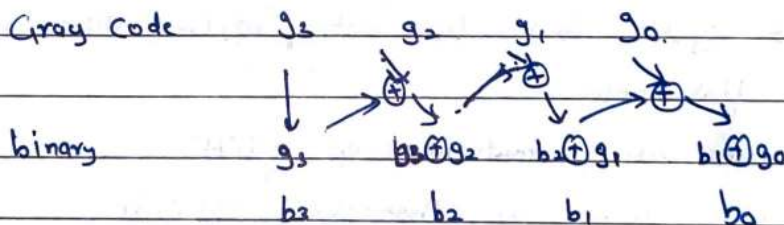
PAGE NO.:

Ex: $m=29 = (11101)_2$
 $= \underline{1} \oplus 1 \oplus 1 \oplus 0 \oplus 1$
 $= \underline{10011}$
 $\rightarrow$ Gray code of 29.

So.



Gray Code to Binary No.:



Ex: Gray code is 10011

Binary is $1 \oplus 0 \oplus 0 \oplus 0 \oplus 1$
 $= \underline{11101}$
 $\rightarrow 29$

Now we know Decimal $\leftrightarrow$ Gray.

- $\rightarrow$ Gray of m & Gray of $m+1$ has only 1 bit changed.
- $\rightarrow$ No code is wasted here (all per. are used).

* Another way to find Gray codes:

$$n^{\text{th}} \text{ gray code is } n \oplus \left\lfloor \frac{n}{2} \right\rfloor$$

↓
bitwise
XOR.

Ex: Gray code of 29

$$n = 29 = 11101$$

$$\left\lfloor \frac{n}{2} \right\rfloor = 14 = 01110$$

$$\begin{array}{r} 01110 \\ 10011 \\ \hline \end{array}$$

→ Gray code of 29.

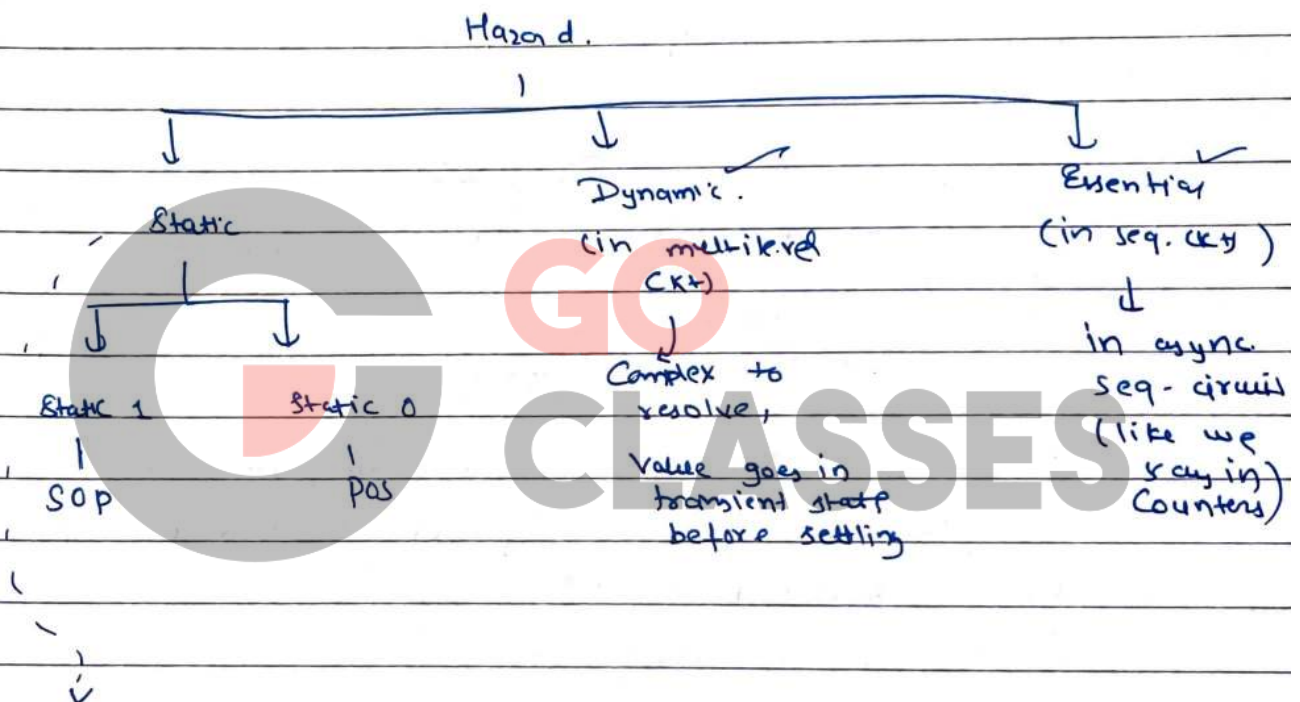
* Properties of Gray Codes:

- Hamming distance is 1 i.e. Gray code differ by 1 bit only
- Code is cyclic bcz. last entry of G_n differ by 1 bit from first entry.
- G_n uses all permutations of n bits
- G_n is same as first half of G_{n+1}

★ Hazards in DL

→ whenever unwanted (or undesirable) transitions in the O/P signal of digital circuit, then we call them hazard.

For Ex: Glitches we saw in async. counter.



★★ Static Hazard: Those hazard when signal level should have been constant BUT it changes for a small amount of time.

For Ex: If signal is 1, it goes from 1 to 0 for small time, (due to static hazard).
or vice versa.

NOTE In any ckt either both static-1 & static-0 hazard will be there or none. (Any one only not possible).

DATE: / /

PAGE NO.:

* Static-1 Hazard: It come in SOP terms.

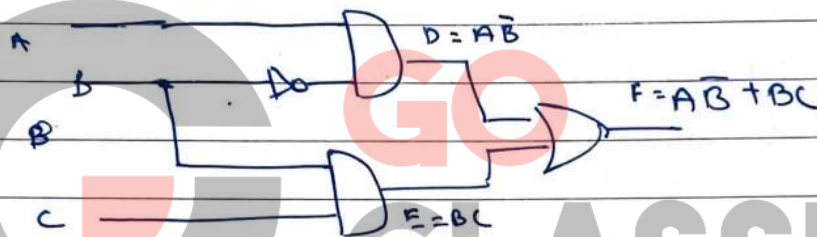
for F_1 .

$$F(A,B,C) = \Sigma(3,4,5,7)$$

A \ BC	00	01	11	10
0	0	0	1	0
1	1	1	1	0

$$f = A\bar{B} + BC$$

⇓ Circuit



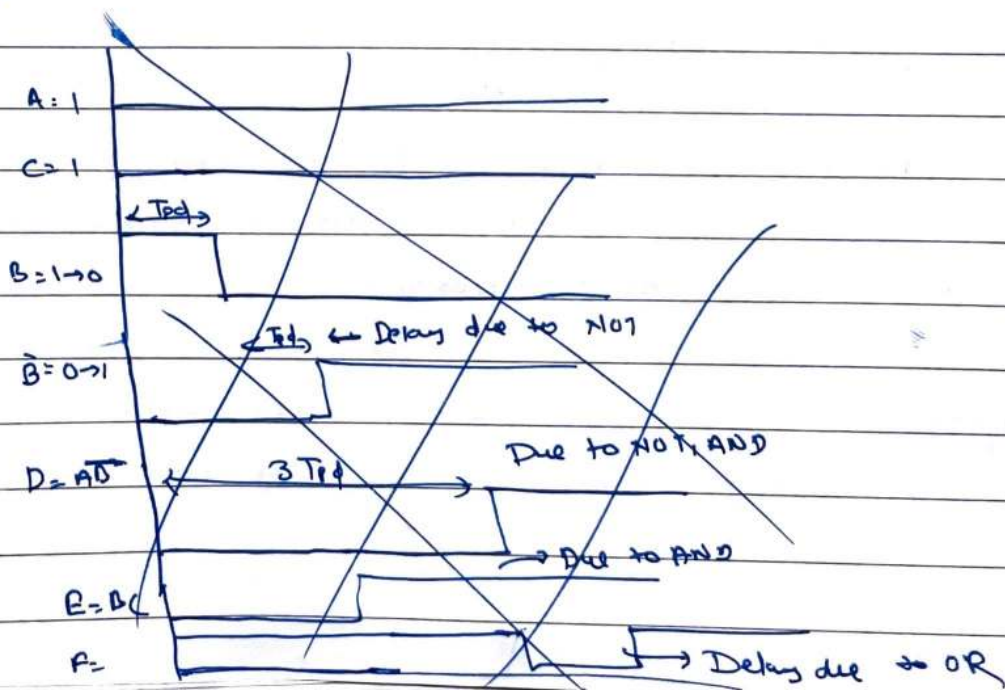
Assume T_{pd} of each gate is same.

initially if $A=C=1$, then $B \rightarrow 1$ to 0 then

Case 1: $A=B=C=1$, $F=1$

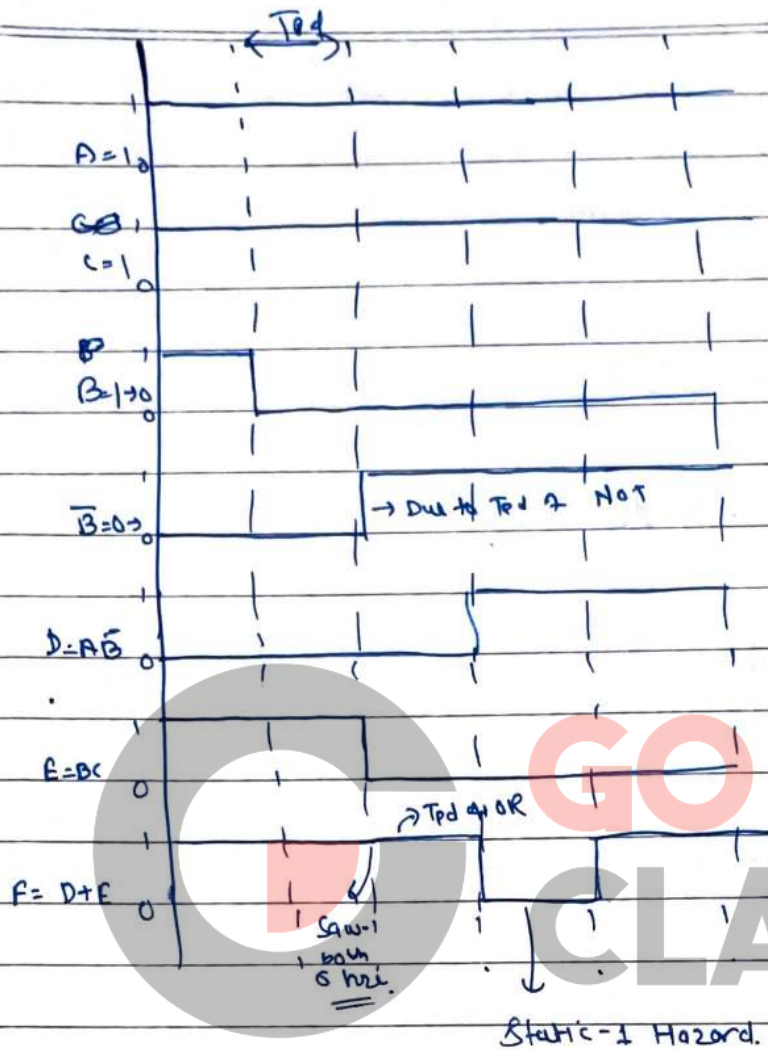
Case 2: $A=C=1$ & $B=0$ then $F=1$

So, in both case $0 \rightarrow 1$ is 1, but See Timing Diagram:



DATE: / /

PAGE NO.:



Static-1 Hazard.

* Static 0 Hazard: Similarly static 0 Hazard occurs in POS terms

Elimination of static hazard:

In prev. ex:

A \ BC				
	00	01	11	10
0	0	0	1	0
1	1	1	0	0

$$f = A\bar{B} + B\bar{C} + AC$$

→ Add this term also,

So we have to add redundant term to remove static hazard.

Similar Consensus Theorem

DATE: / /

PAGE NO.:

→ If there exist a pair of 1's which are not in same cube, it means there is static hazard & each such pair is static 1 hazard.

Ques: Adding which term in below circuit will remove static-1 hazard?

$$F = XY + YZ$$

⇒ Ans = ~~XY~~ XZ ✓

→ Gate CSE 2006 PYQ. ←
✓

These notes are the property of GOClasses. Please do not share them without their consent.

For any questions or issues regarding my handwritten notes, feel free to reach out via email at karan757527@gmail.com or DM me on telegram at @karan757527 (or by clicking [here](#)).

I would love to know if you liked my notes. Your feedback and appreciation are invaluable, so don't hesitate to share your thoughts. Click [here](#) to connect with me on LinkedIn or you can find me as @agrawalkaran.

Wishing you success on your journey ahead!

- Karan Agrawal